

LEGAL RAG APP

دليل البنية البرمجية، قواعد البيانات، المصادقة، والعزل بين المؤسسات

من SQLAlchemy Schemas و Pydantic إلى Logic Models و JWT و AuthController

هدف الدليل

شرح وظيفة كل طبقة في المشروع، علاقتها بالطبقات الأخرى، وكيف تتحرك البيانات بأمان من طلب API حتى PostgreSQL و Vector DB.

إعداد خاص بمشروع Legal RAG App

الإصدار: أغسطس 2026

خريطة الدليل

المحتوى	القسم
الصورة العامة والفرق بين أنواع الـ Schemas والـ Models	1
SQLAlchemy ORM Schemas والجداول والعلاقات	2
Pydantic Schemas للـ Requests والـ Responses	3
Tenant Isolation و Logic Models	4
Controllers ومسارات الملفات والمعالجة	5
Celery و Idempotency ومعالجة الملفات	6
Pagination	7
PasswordHasher	8
JWTService	9
AuthController	10
رحلات التنفيذ الكاملة	11
قائمة فحص الأمان والخطوات التالية	12

1. الصورة العامة للمشروع

المشروع مبني على فصل واضح للمسؤوليات. كل طبقة تقوم بدور محدد، وبالتالي يصبح الكود أسهل في الاختبار والتطوير وأكثر أمانًا.

```
01 HTTP Request
02   -> Pydantic Schema
03   -> Route / Dependency
04   -> Controller
05   -> Logic Model
06   -> SQLAlchemy ORM
07   -> PostgreSQL
08
09 Background Task
10   -> Celery
11   -> file_processing workflow
12   -> Logic Models + Vector DB
```

المسؤولية	الطبقة
تعريف الجداول والأعمدة والعلاقات والقيود في PostgreSQL.	SQLAlchemy ORM Schema
فحص شكل بيانات API وتحديد Request و Response ومنع الحقول غير المصرح بها.	Pydantic Schema
الاستعلامات وعمليات CRUD وقواعد قاعدة البيانات و Tenant Isolation.	Logic Model
تنسيق رحلة العمل بين Models والخدمات الخارجية.	Controller
Hash لكلمات المرور وإنشاء وفحص JWT.	Security Services
استقبال HTTP، استخراج المستخدم الحالي، وفرض الصلاحيات.	Routes / Dependencies

قاعدة مهمة

Pydantic تتحقق من شكل البيانات. Logic Models تتحقق من قواعد قاعدة البيانات والانتماء للمؤسسة. Database Constraints تمثل خط الدفاع النهائي.

2. SQLAlchemy ORM Schemas

ORM Schemas تمثل جداول PostgreSQL. كل Class يرث من SQLAlchemyBase، وكل Attribute من نوع Column يمثل عمودًا في الجدول.

العلاقات	أهم الحقول	الفكرة	الجدول	Schema
Users, Roles, Projects	tenant_id, tenant_name, tenant_code, tenant_status, tenant_settings	المؤسسة أو العميل المستقل في النظام. تمتلك Users و Roles و Projects.	tenants	Tenant
Tenant, UserRoles, Roles	user_id, tenant_id, user_email, password_hash, user_status, is_tenant_admin	المستخدم داخل مؤسسة واحدة. يحتفظ بالبريد Password Hash والحالة.	users	User
Tenant, Users عبر	role_id, tenant_id,	دور أو مجموعة	roles	Role

		صلاحيات داخل مؤسسة، مثل Tenant Admin أو Viewer.	role_name, role_description, is_system_role	UserRole
UserRole	user_roles	جدول ربط Many-to-Many بين Users و Roles.	user_id, role_id, assigned_at	User, Role
Project	projects	مساحة عمل أو Knowledge Base داخل Tenant.	project_id, project_uuid, tenant_id	Tenant, Assets, Chunks
Asset	assets	ملف مرفوع داخل Project مع حالة المعالجة وعدد Chunks.	asset_id, asset_uuid, asset_type, asset_status, asset_project_id	Project, Chunks
DataChunk	chunks	جزء نصي مستخرج من Asset ويستخدم في الفهرسة والاسترجاع.	chunk_id, chunk_uuid, chunk_text, metadata, order, project_id, asset_id	Project, Asset
CeleryTaskExecution	celery_task_executions	سجل تنفيذ مهمة خلفية وحالتها ونتيجتها أو خطئها.	execution_id, task_name, args_hash, status, result, timestamps	يستخدمه IdempotencyManager

2.1 العلاقات الأساسية

```

01 Tenant 1 ---- * Users
02 Tenant 1 ---- * Roles
03 Tenant 1 ---- * Projects
04 User * ---- * Roles (through UserRole)
05 Project 1 --- * Assets
06 Project 1 --- * Chunks
07 Asset 1 --- * Chunks

```

- ForeignKey يحدد الارتباط بين الجدولين.
- "onDelete="CASCADE" يحذف السجلات التابعة عند حذف السجل الأب، لذلك حذف Tenant أو Project عملية حساسة.
- UniqueConstraint مثل tenant_id + user_email يمنع تكرار البريد داخل نفس المؤسسة فقط.
- CheckConstraint يمنع حالات غير معروفة مثل status غير active أو inactive أو suspended.
- __repr__ يحدد شكل Object أثناء الطباعة والـ debugging، ويجب أن يرجع String دون فاصلة أخيرة.

Pydantic Schemas .3

Pydantic هي بوابة التحقق من بيانات API. لا تتصل بقاعدة البيانات، لكنها تمنع بيانات غير صحيحة أو حقول إضافية قبل الوصول إلى Controller.

الاستخدام	Schemas الأساسية	الملف
إنشاء وتعديل وإرجاع المؤسسات.	TenantCreate, TenantUpdate, TenantResponse, TenantListResponse	tenant.py
إدارة المستخدمين دون كشف password_hash.	UserCreate, UserUpdate, UserPasswordUpdate, UserResponse, UserListResponse	user.py
إدارة الأدوار داخل Tenant.	RoleCreate, RoleUpdate, RoleResponse, RoleListResponse	role.py
إسناد وإزالة أدوار المستخدم.	UserRoleAssign, BulkAssign, Responses	user_role.py
تسجيل الدخول والتوكن والمستخدم الحالي.	LoginRequest, RefreshTokenRequest, TokenResponse, Token Payloads, CurrentUserResponse	auth.py
استجابات ومعاملات مشتركة.	MessageResponse, ErrorResponse, PaginationParams	common.py

3.1 أهم إعدادات Pydantic

```
01 class RoleCreate(BaseModel):
02     model_config = ConfigDict(
03         extra="forbid",
04         str_strip_whitespace=True,
05     )
06
07     role_name: str = Field(
08         ...,
09         min_length=2,
10         max_length=100,
11     )
```

- "extra="forbid" يمنع إرسال حقول غير معرفة، مثل is_system_role من عميل عادي.
- str_strip_whitespace=True يزيل المسافات من بداية ونهاية النصوص.
- Field يحدد الطول والقيم الافتراضية والأمثلة والحدود الرقمية.
- field_validator ينظف حقلاً واحداً، و model_validator يقارن أكثر من حقلاً، مثل منع تساوي كلمة المرور الجديدة والقديمة.
- ConfigDict(from_attributes=True) يسمح بتحويل SQLAlchemy Object مباشرة إلى Response آمن.

فصل مهم

UserCreate يستقبل password مؤقتًا، بينما UserResponse لا يحتوي password أو password_hash نهائيًا.

4. Logic Models

Logic Models تتعامل مع Session Factory بالشكل `async with self.db_client() as session`. وهي مسؤولة عن استعلامات قاعدة البيانات والعزل بين المؤسسات.

Logic Model	وظيفته
TenantModel	إنشاء المؤسسة، البحث بالكود أو ID، القائمة، التعديل، الحذف، exists و is_active.
UserModel	إنشاء المستخدم، Login lookup داخل Tenant، تعديل الحساب، Password Hash، last_login، الحذف.
RoleModel	إنشاء وعرض وتعديل وحذف Roles مع منع حذف System Roles.
UserRoleModel	إسناد Role واحدة أو عدة Roles، الفحص، العرض، العد، والإزالة.
ProjectModel	إنشاء وعرض وحذف Projects داخل Tenant، والبحث بـ ID أو UUID.
AssetModel	إنشاء ملف، البحث، Pagination، حالات uploaded/processing/indexed/failed، الحذف.
ChunkModel	إنشاء Chunks فردية أو Batch، العرض والعد والحذف داخل Tenant.

4.1 Tenant Isolation

العزل يعني أن مستخدم Tenant A لا يستطيع قراءة أو تعديل بيانات Tenant B حتى لو عرف ID صحيحًا. لذلك يجب أن تحتوي كل Query حساسة على tenant_id أو Join إلى Project ثم فلترة Tenant.

```
01 query = select(User).where(  
02     User.user_id == user_id,  
03     User.tenant_id == tenant_id,  
04 )
```

```
01 query = (
```

```

02     select(Asset)
03     .join(Project)
04     .where(
05         Asset.asset_id == asset_id,
06         Project.tenant_id == tenant_id,
07     )
08 )

```

ممنوع

لا نأخذ tenant_id من Request Body في العمليات العادية. يتم استخراجه من المستخدم الحالي بعد التحقق من Access Token.

5. Controllers

الدور	Controller
إنشاء مسارات الملفات بالشكل files/tenant_id/project_id ومنع Path Traversal.	ProjectController
فحص نوع وحجم الملف، تنظيف الاسم، وإنشاء اسم ومسار فريد.	DataController
قراءة PDF/TXT، تنظيف النص والـ metadata، تقسيم النص إلى Chunks، وإضافة tenant_id للـ metadata.	ProcessController
تنسيق Login و Refresh و me باستخدام Models وخدمات الأمان.	AuthController
التعامل مع Embeddings و Vector DB والبحث والتوليد، وتتم مراجعته قبل تشغيل RAG متعدد المؤسسات.	NLPController

5.1 رحلة الملف

```

01 UploadFile
02     -> DataController validates and stores file
03     -> Asset record: uploaded
04     -> Celery task starts
05     -> ProcessController extracts and cleans text
06     -> ChunkModel stores chunks
07     -> NLPController creates embeddings
08     -> Vector DB indexes vectors
09     -> Asset status: indexed

```

6. Celery, Idempotency

Celery تنفذ المهام الثقيلة خارج Request الأساسي. celery_app.py يحدد Backend و Broker و Queue وال time limit، بينما file_processing.py ينظم رحلة معالجة الملف.

6.1 IdempotencyManager

- ينشئ SHA-256 Hash من task_name و task_args بعد ترتيب مفاتيح JSON.
- يمنع تشغيل نفس المهمة مرتين إذا كانت PENDING أو STARTED أو SUCCESS.
- يسمح بإعادة المحاولة عند FAILURE أو عند اكتشاف مهمة عالقة بعد المهلة.
- يسجل started_at و completed_at و result و error_message.
- يدخل tenant_id ضمن task_args، وبالتالي لا تختلط مهام المؤسسات.

```
01 task_args = {
02     "tenant_id": str(tenant_id),
03     "project_id": project_id,
04     "asset_id": asset_id,
05     "file_id": file_id,
06 }
```

7. Pagination

Pagination تقسم النتائج الكبيرة إلى صفحات بدل تحميل كل السجلات مرة واحدة. هذا يقلل استهلاك الذاكرة وحجم الاستجابة والضغط على PostgreSQL.

```
01 offset = (page - 1) * page_size
02
03 query = (
04     select(User)
05     .offset(offset)
06     .limit(page_size)
07 )
```

مثال: 25 مستخدمًا و page_size=10 يعني 3 صفحات. الصفحة الثانية تتخطى أول 10 سجلات وتعيد السجلات من 11 إلى 20.

```
01 total_pages = (
02     total_records + page_size - 1
03 ) // page_size
```

المعنى	القيمة
رقم الصفحة الحالية، ويبدأ من 1.	page
عدد العناصر في الصفحة، مع حد أقصى مثل 100.	page_size
إجمالي السجلات المطابقة.	total

total_pages	إجمالي عدد الصفحات.
offset	عدد السجلات التي يتم تخطيها.

8. PasswordHasher

PasswordHasher يحول كلمة المرور الأصلية إلى Hash باستخدام Argon2، ثم يتحقق منها وقت Login. لا يتم تخزين كلمة المرور الأصلية ولا يمكن فك ال Hash إلى الأصل.

```
01 password_hash = password_hasher.hash_password(
02     plain_password
03 )
04
05 is_valid = password_hasher.verify_password(
06     plain_password,
07     password_hash,
08 )
```

- Argon2 يضيف Salt عشوائيًا، ولذلك قد تنتج نفس كلمة المرور Hash مختلفًا كل مرة.
- UserModel يستقبل password_hash فقط. Controller هو المسؤول عن استخدام PasswordHasher أولاً.
- UserResponse لا يعرض password ولا password_hash.

تدفق إنشاء المستخدم

UserCreate.password -> PasswordHasher -> User.password_hash -> UserModel.create_user() -> PostgreSQL

9. JWTService

JWTService يصدر Access Token و Refresh Token ويتحقق من التوقيع والصلاحيات ونوع التوكن وال claims المطلوبة.

Claim	وظيفته
sub	معرف المستخدم user_id.
tenant_id	المؤسسة التي ينتمي إليها المستخدم.
token_type	access أو refresh.
roles	أسماء الأدوار داخل Access Token.
is_tenant_admin	هل المستخدم مدير للمؤسسة.

iat	وقت إصدار التوكن.
exp	وقت انتهاء الصلاحية.
jti	معرف فريد للتوكن.

9.1 Access مقابل Refresh

Access Token	Refresh Token
قصيرة العمر، مثل 30 دقيقة.	أطول عمرًا، مثل 7 أيام.
تُرسل مع كل API محمية.	تُستخدم فقط لإصدار Token Pair جديدة.
تحتوي roles و is_tenant_admin.	لا تحتاج Roles لأننا نعيد تحميلها من DB.
Authorization: Bearer TOKEN	POST /auth/refresh

```

01 token_pair = jwt_service.create_token_pair(
02     user_id=user.user_id,
03     tenant_id=user.tenant_id,
04     is_tenant_admin=user.is_tenant_admin,
05     roles=role_names,
06 )

```

تنبيه

JWT موقعة وليست مكانًا للأسرار. لا نضع password أو password_hash داخل Payload، ولا نرفع JWT_SECRET_KEY إلى Git.

10. AuthController

AuthController هو منسق المصادقة. يستخدم UserModel و TenantModel و PasswordHasher و JWTService بدل تنفيذ SQL أو Hash أو JWT بنفسه.

10.1 login()

```

01 tenant_code + email + password
02     -> find user and active tenant
03     -> verify password hash
04     -> load roles
05     -> issue access + refresh tokens
06     -> update last_login_at

```

- يستخدم رسالة invalid credentials لجميع حالات الفشل لتقليل تسريب معلومات الحسابات.
- يتأكد من user_status و tenant_status قبل إصدار التوكن.
- يحمل Roles ويضع أسماءها داخل Access Token.

10.2 refresh_tokens()

- يفك Refresh Token ويتأكد أن token_type هو refresh.
- يعيد تحميل المستخدم و Tenant و Roles من قاعدة البيانات.
- يصدر Token Pair جديدة، وبالتالي تظهر أحدث الصلاحيات.

10.3 get_current_user()

- يفك Access Token ويستخرج user_id و tenant_id.
- جلب بيانات المستخدم والمؤسسة والأدوار الفعلية من قاعدة البيانات.
- يرجع CurrentUserResponse دون بيانات حساسة.

11. رحلات التنفيذ الكاملة

11.1 إنشاء مستخدم

```
01 POST /users
02     -> UserCreate validates input
03     -> current_user.tenant_id determines tenant
04     -> PasswordHasher creates hash
05     -> User ORM object
06     -> UserModel.create_user()
07     -> UserResponse
```

11.2 تسجيل الدخول

```
01 POST /auth/login
02     -> LoginRequest
03     -> AuthController.login()
04     -> UserModel lookup
05     -> PasswordHasher.verify_password()
06     -> Tenant active check
07     -> JWTService.create_token_pair()
08     -> TokenResponse
```

11.3 طلب محمي

```
01 Authorization: Bearer ACCESS_TOKEN
02     -> Auth dependency decodes token
03     -> loads active user and tenant
04     -> route gets current_user
05     -> tenant_id comes from current_user
06     -> Logic Model applies tenant isolation
```

11.4 رفع ومعالجة ملف

```
01 Authenticated upload
02   -> tenant-aware storage path
03   -> Asset record
04   -> Celery task with tenant_id
05   -> idempotency check
06   -> extract + chunk
07   -> PostgreSQL chunks
08   -> Vector DB indexing
09   -> Asset status indexed
```

12. قائمة فحص الأمان

- ☐ عدم تخزين أو طباعة كلمات المرور الأصلية أو Password Hash في Responses أو Logs.
- ☐ استخراج tenant_id من Access Token وليس من Request Body.
- ☐ إضافة tenant_id إلى كل Query أو Join حساس.
- ☐ منع حذف System Roles وحماية حذف Tenant وProject بصلاحيات مرتفعة.
- ☐ استخدام Access Token فقط للـ APIs وRefresh Token فقط للتجديد.
- ☐ حفظ JWT_SECRET_KEY في البيئة وعدم رفعه إلى Git.
- ☐ إضافة tenant_id إلى Vector metadata أو Namespace لمنع اختلاط نتائج RAG.
- ☐ تمرير tenant_id إلى Celery task وإدخاله في Idempotency Hash.
- ☐ حماية مسارات الملفات من Path Traversal.
- ☐ استخدام Database Constraints حتى لو كان Pydantic موجودًا.

13. هيكل الملفات النهائي المقترح

```
01 src/
02 |-- core/
03 |   |-- password_hasher.py
04 |   |-- jwt_service.py
05 |-- schemas/
06 |   |-- tenant.py
07 |   |-- user.py
08 |   |-- role.py
09 |   |-- user_role.py
10 |   |-- auth.py
11 |   |-- common.py
12 |-- models/
13 |   |-- TenantModel.py
14 |   |-- UserModel.py
15 |   |-- RoleModel.py
16 |   |-- UserRoleModel.py
17 |   |-- ProjectModel.py
18 |   |-- AssetModel.py
19 |   |-- ChunkModel.py
20 |-- controllers/
21 |   |-- AuthController.py
22 |   |-- ProjectController.py
23 |   |-- DataController.py
24 |   |-- ProcessController.py
25 |   |-- NLPController.py
26 |-- dependencies/
27 |   |-- auth.py
28 |-- routes/
29 |   |-- auth.py
30 |-- tasks/
31 |   |-- file_processing.py
32 |-- utils/
33 |   |-- idempotency_manager.py
```

14. الخطوات التالية

1. إنشاء Auth Dependencies: `get_current_user` و `require_role` و `require_tenant_admin`.
2. إنشاء Auth Routes: `login` و `refresh` و `me`.
3. إنشاء Seed Script لأول Tenant و Admin User و System Roles.
4. تحديث Routes القديمة لتمرير `tenant_id` من `current_user`.
5. مراجعة NLPController و Vector DB لضمان Tenant Isolation داخل البحث والفهرسة.
6. إضافة Tests للمصادقة والعزل بين المؤسسات وحالات Celery.

الخلاصة

المشروع الآن مبني على سلسلة واضحة: Pydantic يفحص البيانات، Controllers تنظم الرحلات، Logic Models تنفذ قواعد قاعدة البيانات، PasswordHasher يحمي كلمات المرور، JWTService يدير التوكنز، و AuthController يربط عملية المصادقة بالكامل.